Algorithms

MINIMUM SPANNING TREES (PART 2)

SOME SLIDES ARE TAKEN FROM ALGORITHMS COURSE, TECHNION

Last Week

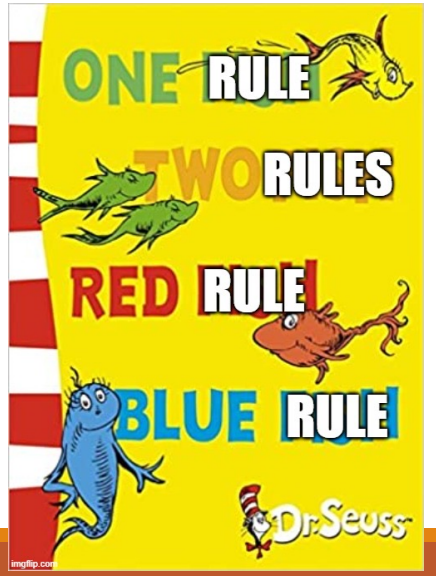
- Minimum Spanning Trees (MST)
  - Generic rules (Red rule, Blue rule)
  - Kruskal Algorithm

This Week

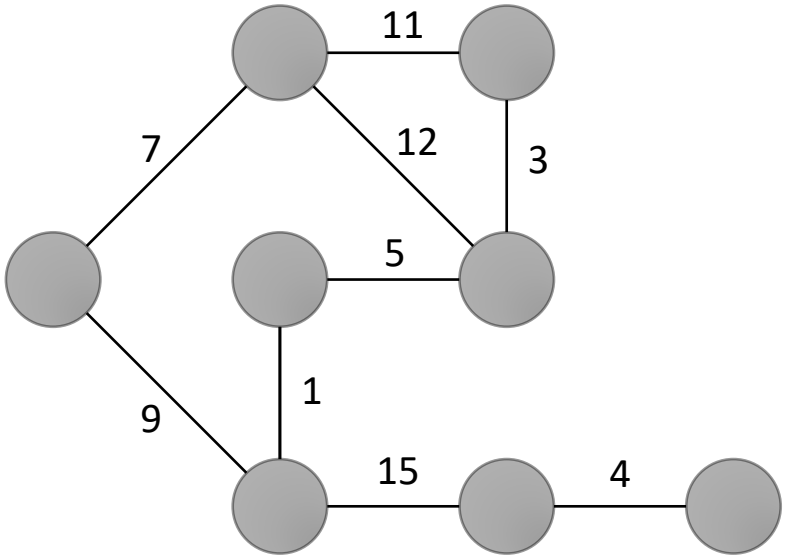
- Minimum Spanning Trees (MST)
  - Prim Algorithm
  - MST Questions (including Kruskal)

Reminder

1. Blue Rule: Let U be a **cut** with no blue edges. Select an uncolored edge in U of minimum weight and color it blue.
2. Red Rule: Let C be a **cycle** with no red edges. Select an uncolored edge of C of maximum weight and color it red.



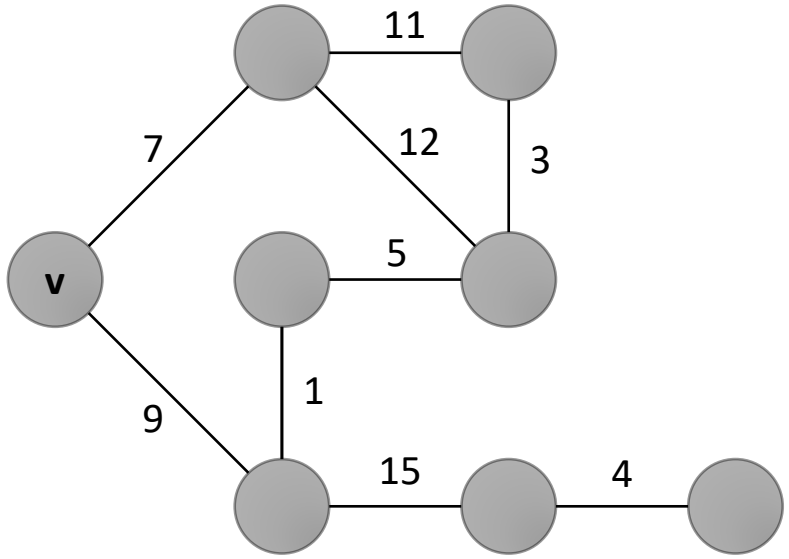
Prim Algorithm



Algorithm 1 PRIM(G,w)

$v \leftarrow$ a random vertex
$U \leftarrow \{v\}$
While U does not span G:
    $C \leftarrow$ cut induced by U
    $e \leftarrow$ minimum-weight edge $\in C$
    $U \leftarrow U \cup \{\text{endpoint of } e \text{ in } V - U\}$

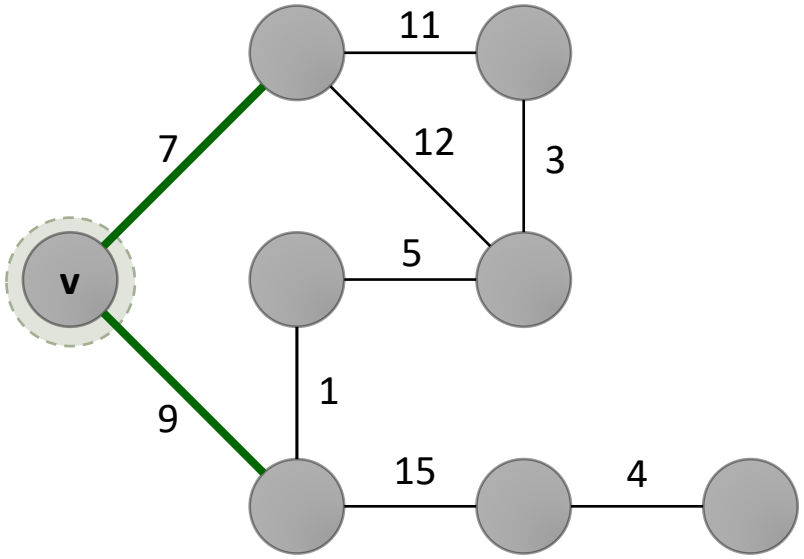
Prim Algorithm



Algorithm 1 PRIM(G,w)

$v \leftarrow$ a random vertex
$U \leftarrow \{v\}$
While U does not span G:
    $C \leftarrow$ cut induced by U
    $e \leftarrow$ minimum-weight edge $\in C$
    $U \leftarrow U \cup \{\text{endpoint of } e \text{ in } V - U\}$

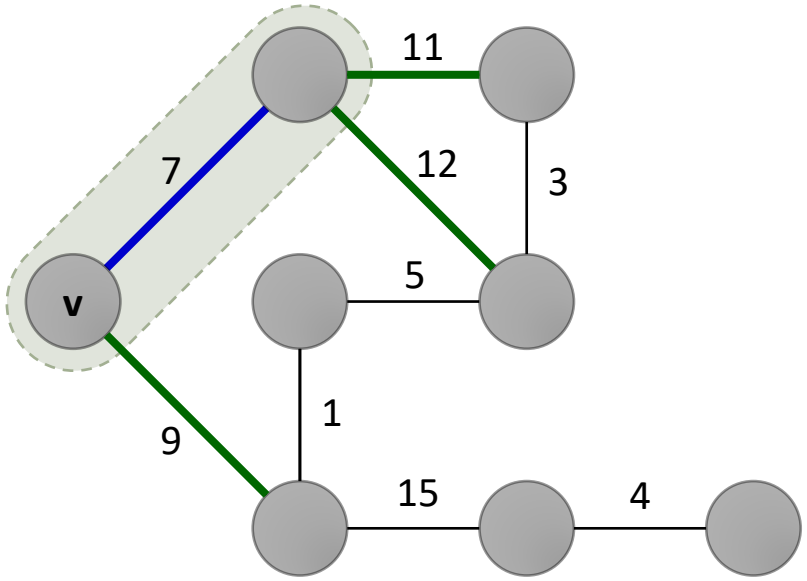
Prim Algorithm



Algorithm 1 PRIM(G,w)

$v \leftarrow$ a random vertex
$U \leftarrow \{v\}$
While U does not span G:
    $C \leftarrow$ cut induced by U
    $e \leftarrow$ minimum-weight edge $\in C$
    $U \leftarrow U \cup \{\text{endpoint of } e \text{ in } V - U\}$

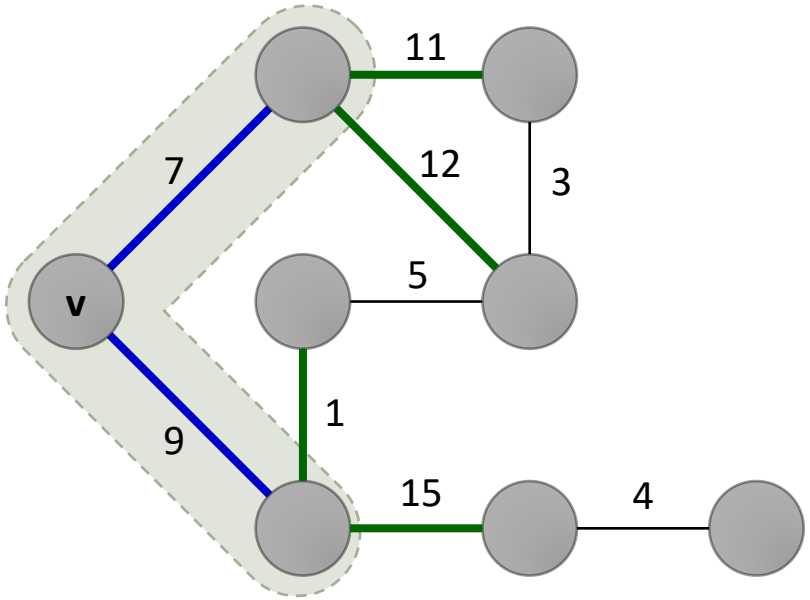
Prim Algorithm



Algorithm 1 PRIM(G,w)

$v \leftarrow$ a random vertex
$U \leftarrow \{v\}$
While U does not span G:
    $C \leftarrow$ cut induced by U
    $e \leftarrow$ minimum-weight edge $\in C$
    $U \leftarrow U \cup \{\text{endpoint of } e \text{ in } V - U\}$

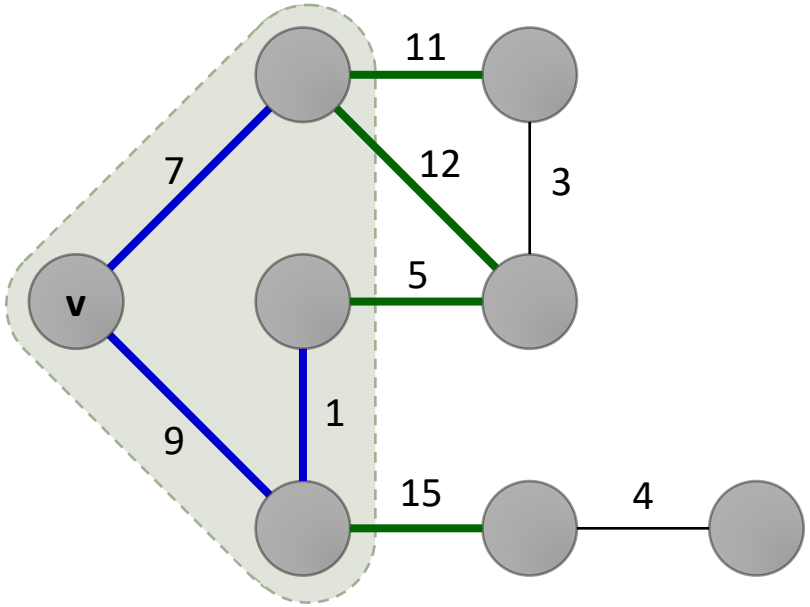
Prim Algorithm



Algorithm 1 PRIM(G,w)

$v \leftarrow$ a random vertex
$U \leftarrow \{v\}$
While U does not span G:
    $C \leftarrow$ cut induced by U
    $e \leftarrow$ minimum-weight edge $\in C$
    $U \leftarrow U \cup \{\text{endpoint of } e \text{ in } V - U\}$

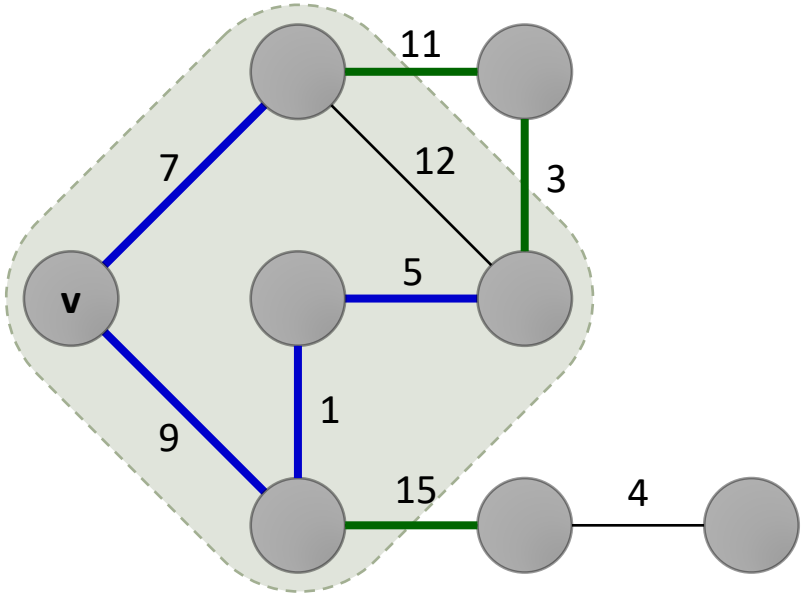
Prim Algorithm



Algorithm 1 PRIM(G,w)

$v \leftarrow$ a random vertex
$U \leftarrow \{v\}$
While U does not span G:
    $C \leftarrow$ cut induced by U
    $e \leftarrow$ minimum-weight edge $\in C$
    $U \leftarrow U \cup \{\text{endpoint of } e \text{ in } V - U\}$

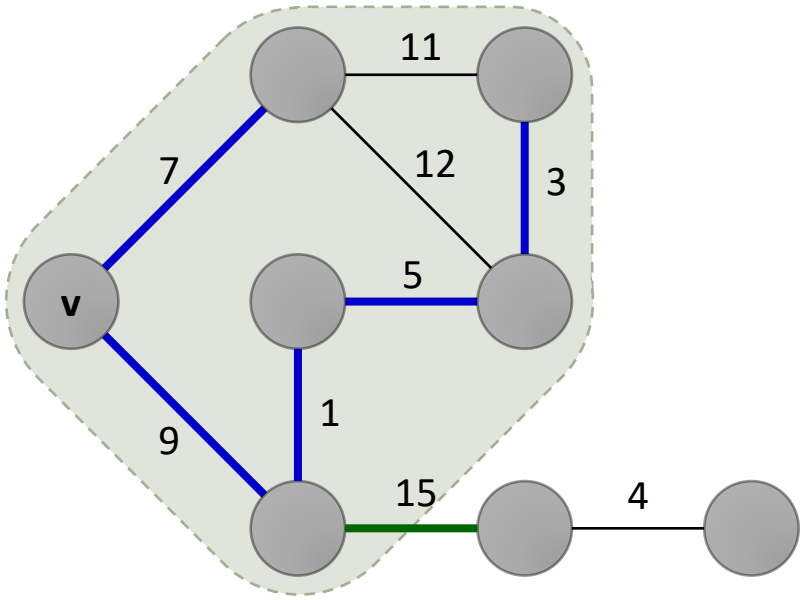
Prim Algorithm



Algorithm 1 PRIM(G,w)

$v \leftarrow$ a random vertex
$U \leftarrow \{v\}$
While U does not span G:
    $C \leftarrow$ cut induced by U
    $e \leftarrow$ minimum-weight edge $\in C$
    $U \leftarrow U \cup \{\text{endpoint of } e \text{ in } V - U\}$

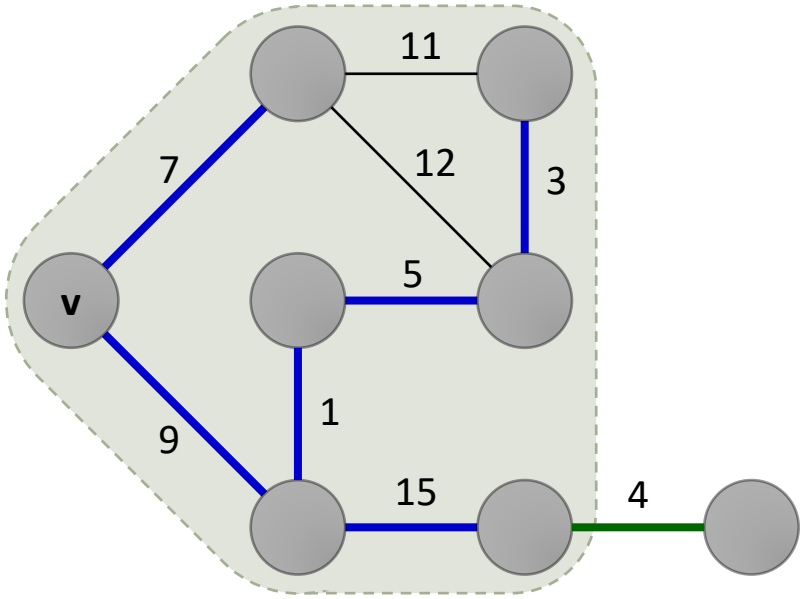
Prim Algorithm



Algorithm 1 PRIM(G,w)

$v \leftarrow$ a random vertex
$U \leftarrow \{v\}$
While U does not span G:
    $C \leftarrow$ cut induced by U
    $e \leftarrow$ minimum-weight edge $\in C$
    $U \leftarrow U \cup \{\text{endpoint of } e \text{ in } V - U\}$

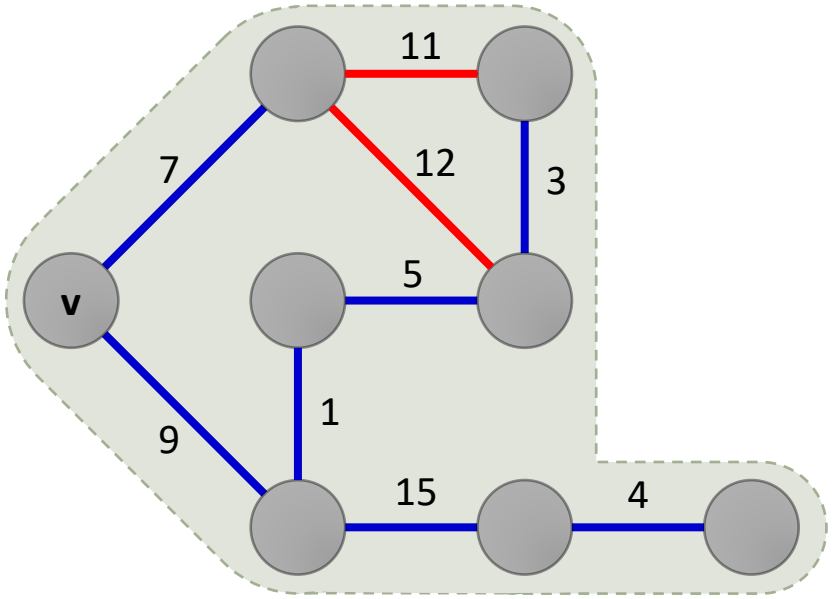
Prim Algorithm



Algorithm 1 PRIM(G,w)

$v \leftarrow$ a random vertex
$U \leftarrow \{v\}$
While U does not span G:
    $C \leftarrow$ cut induced by U
    $e \leftarrow$ minimum-weight edge $\in C$
    $U \leftarrow U \cup \{\text{endpoint of } e \text{ in } V - U\}$

Prim Algorithm



Algorithm 1 PRIM(G,w)

$v \leftarrow$ a random vertex
$U \leftarrow \{v\}$
While U does not span G:
    $C \leftarrow$ cut induced by U
    $e \leftarrow$ minimum-weight edge $\in C$
    $U \leftarrow U \cup \{\text{endpoint of } e \text{ in } V - U\}$

Prim Algorithm

Correctness:

The algorithm repeatedly executes the blue rule n-1 times. In each step we consider the cut defined by the vertices that are already in the tree.

Prim Algorithm

Data Structures in Use:

- The graph is implemented with adjacency lists.
- A 'best-edge' array that keeps for each vertex not in the tree the cheapest edge that connects the vertex with the tree (if such exists) or infinity otherwise.
- A priority queue whose keys are the costs of edges in the ‘best-edge’ array.

Prim Algorithm

Time Complexity:

- Initiate a (priority) queue with all vertices of G takes $O(n)$.
- A vertex is dequeued when it joins the tree (single dequeue - $O(logn)$). Since every vertex joins the tree once, there are n dequeue operations. In total: $O(nlogn)$.
- Every edge $e = (u, v)$ is considered twice; when its endpoints join the tree. Each time an edge is considered at most one decrease key operation is performed. In total: $O(mlogn)$.
- Total time is $O(mlogn + nlogn) = O(mlogn)$
- Actually $O(m + nlogn)$ if using Fibonacci heap.

Exercise 1

Let G = (V, E) be an undirected, connected weighted graph. Suggest an algorithm that determines whether an edge $e = (a, b) \in E$ appears in *some* MST of G.

Exercise 1 – Solution 1

- Color e yellow and run the algorithm to find the most yellow MST (that is, in the sorted list of edges, e is first among edges having cost c(e))
- If $e \in T$, return “e is in some MST”.
- Else, return “e is not in any MST”.

Exercise 1 – Solution 1

Correctness:

If a MST that contains e exists, then e is part of a most-yellow MST.

Exercise 1 – Solution 1

Time Complexity:

The complexity of running Kruskal algorithm - $O(mlogn)$.



Exercise 1 – Solution 2

We will use the following property shown in the lecture:

Let G be a connected undirected graph with positive edge weights. It is known that for any two vertices $a, b \in V$, if there exists a path connecting a and b consisting of edges having weight less than c, then in every MST of G, the (only) path connecting a and b consists of edges having weight less than c.

Exercise 1 – Solution 2

Algorithm:

1. Remove from the graph all the edges of weight at least w(e) (including e). Let G' denote the resulting graph (note that G' contains only edges with weights $< w(e)$).
2. Assume $e = (a, b)$. Use DFS or BFS in order to determine whether a and b are connected in G' and return YES if and only if they are NOT.

Exercise 1 – Solution 2

Correctness:

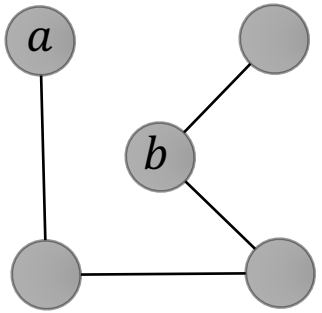
By BFS/DFS. a and b are in different components of G' if and only if there is no path consisting of edges having weight less than w(e) connecting them.

Exercise 1 – Solution 2

Correctness:

- Assume that Kruskal's algorithm is performed on the graph. Consider the time after all edges with weight less than w(e) were considered.
- By the property, independent of the order according to which Kruskal processes the edges, if there is a path of cheaper edges connecting a and b, then they are in the same component of the constructed tree, implying that the edge (a, b) closes a cycle and is not part of the MST.

Case 1: a and b are connected by cheaper edges. Tree components after all edges of weights <w(e) are considered:

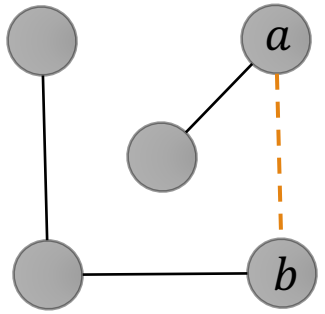


Exercise 1 – Solution 2

Correctness:

- If there is no path of cheaper edges connecting them, then if Kruskal considers (a, b) as the first edge of cost w(e), the edge (a, b) does not close a cycle and is included in some MST.

Case 2: a and b are not connected by cheaper edges.
Tree components after all edges of weights <w(e) are considered:



Exercise 1 – Solution 2

Time Complexity:

- Step 1: Assume the graph is given by adjacency lists: $O(m)$.
  - Connected graph $m \geq \mathrm{n} - 1$
- Step 2: DFS or BFS: $O(m)$.
- All together: $O(m)$.

Exercise 2

Let G=(V, E) be an undirected, connected and weighted graph. Suggest an algorithm that finds a maximum spanning tree in G.

Exercise 2 - Solution

- Intuition: we have an algorithm to find **minimum** spanning tree. Can we use it?
- Algorithm:
  - Let $C > \max_{e} w(e)$.
  - Define a new weight function: $w'(e) = C - w(e)$.
  - Find an MST of G w.r.t. w'.
  - Return the resulting tree.

Exercise 2 - Solution

Correctness:

Let $T = (V, E_T)$ be the output of the algorithm. Then T is MST w.r.t. w'.

Assume towards contradiction that T is not the maximal ST w.r.t. w. Let T' $= (V, E_{T'})$ be some maximum ST such that $w(T) < w(T')$.

By the weight function w' (note that every ST contains $n-1$ edges):

- $w'(T) = (n-1)C - \sum_{e \in E_T} w(e) = (n-1)C - w(T)$
- $w'(T') = (n-1)C - \sum_{e \in E_{T'}} w(e) = (n-1)C - w(T')$

It follows that $w'(T') < w'(T)$, with contradiction to the fact that T is minimum w.r.t w'.

Exercise 3

Let $G = (V, E)$ be an undirected, connected and weighted graph. Find a subset of the edges $E' \subseteq E$ such that $E \backslash E'$ is a ST of G and E' has minimal weight (note that E' is not a ST).

Exercise 3 - Solution

- Run the maximum ST algorithm on G and obtain T. Denote the edges of T by E_T.
- Return $E' = E \backslash E_T$.
- Correctness:
  - $E \backslash E' = E_T$ - the resulting ST.
  - $w(G) = w(E_T) + w(E')$. Since $w(E_T)$ is maximal, then $w(E')$ is minimal.
- Time complexity: Dominated by the MST algorithm,
- $O(m + nlogn)$.

Exercise 4

Let $G = (V, E)$ be an undirected, connected and weighted graph. For every $v \in V$ there is a positive weight $w(v)$.

We want to find a spanning tree T minimizing $\sum_v w(v) \cdot deg_T(v)$ where $deg_T(v)$ is the degree of the vertex v in the spanning tree T.

Exercise 4 - Solution

1. Construct a cost function for the edges c: $\mathrm{E} \rightarrow R^{+}$.

For every edge $e = (v_1, v_2)$, let $\mathrm{c}(e) = w(v_1) + w(v_2)$.

2. Run Prim's algorithm for finding an MST of G with cost function c.

Exercise 4 - Solution

Time Complexity:

Constructing the new weight function takes $O(m)$ time.

Running time is dominated by the MST algorithm, $O(m + nlogn)$.

Exercise 4 - Solution

Proof: Let T be a ST of G.

By the construction, the contribution of every edge $e = (v_1, v_2)$ to cost(T) is $w(v_1) + w(v_2)$.

For every vertex v, the number of edges adjacent to v in T is $deg_T(v)$. Therefore, v contributes exactly $deg_T(v)w(v)$ to cost(T).

Summing up over all vertices, cost(T)= $\sum_v w(v)deg_T(v)$.

In particular, a MST is a spanning tree minimizing $\sum_v w(v)deg_T(v)$.

Exercise 5

Let G = (V, E) be an undirected connected graph with positive weights on the edges. Let $T = (V, E_T)$ be some MST of G.

Let G' = (V, E') such that $E \subseteq E'$ and $|E'| \leq |E| + 2|V|$.

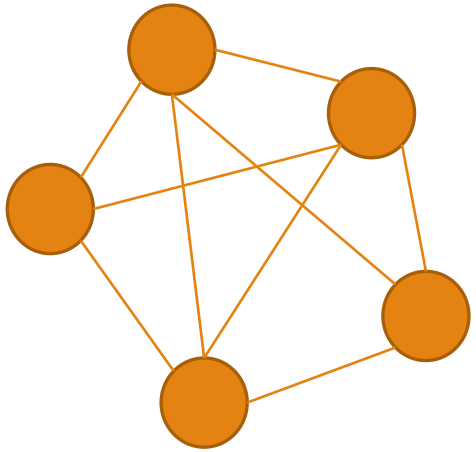
Suggest an algorithm that finds an MST in G' that runs in $O(|V| \log|V|)$.

Note: G' is produced from G by adding at most 2|V| edges.

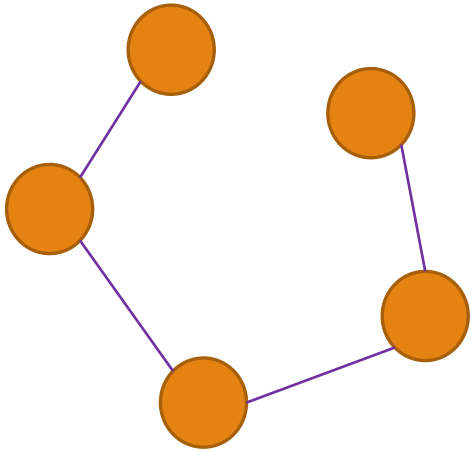
Exercise 5

Example:

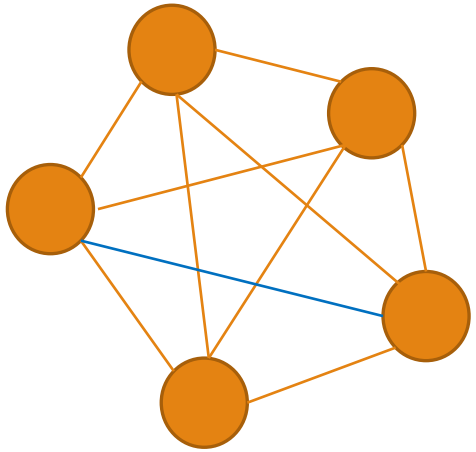
G



T



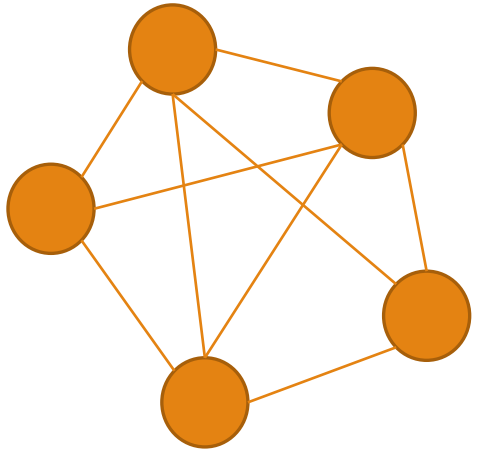
G'



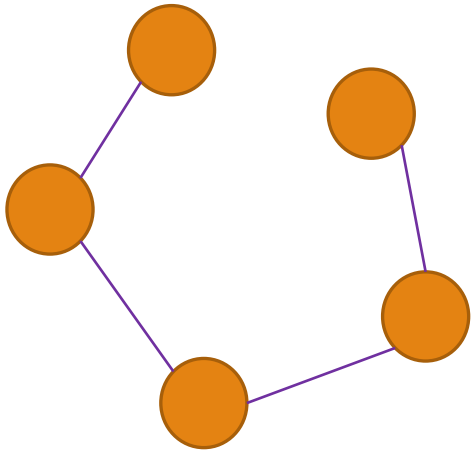
Exercise 5 - Solution

1. Construct a new graph $G'' = (V, E'')$, where $E'' = (E' \backslash \mathrm{E}) \cup E_T$ (namely, all the edges in T and the new edges in G').
2. Run Prim (or Kruskal) on G'' and return the resulting MST.

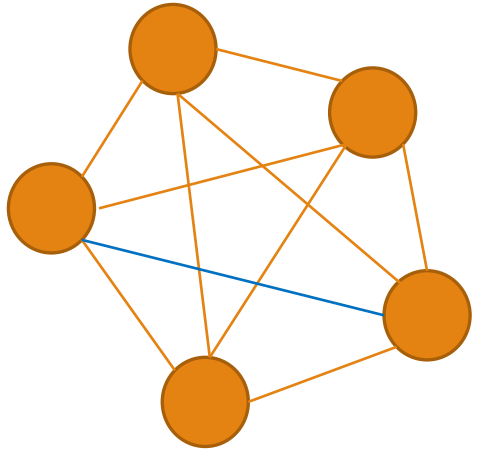
G



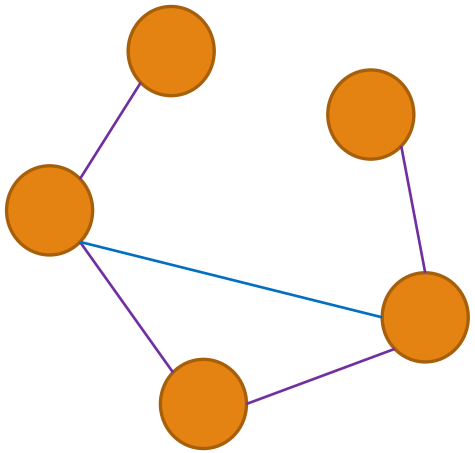
T



G'



G''



Exercise 5 - Solution

Time Complexity:

Bound the number of edges in G'':

- T is a tree $\rightarrow |\mathrm{E_T}| = |V| - 1$
- We know that $|E'| \leq |E| + 2|V|$

$E'' = (E' \backslash \mathrm{E}) \cup E_T$:

$|E''| \leq |E'| - |E| + |E_T| \leq |E| + 2|V| - |E| + |V| - 1 = 3|V| - 1 = O(|V|)$

Running Prim (or Kruskal): since $|E''| = O(|V|)$, this run takes $O(|V| \log |V|)$ as requested

Exercise 5 - Solution

Correctness:

The crucial point: G'' includes only the new edges, and the edges of the MST of G.

We need to show that the set E'' includes an MST of G', i.e. **there exists an MST in G' that doesn't include any edge from $E \backslash E_T$.**

Exercise 5 - Solution

Correctness:

Assume that Kruskal's algorithm is performed on G', breaking ties in favor of edges in E'' (that is, edges in $E \setminus E_T$ are last among edges with their weight).

Consider an edge $e = (u, v) \in E \setminus E_T$. Since T is connected, it includes a path connecting u and v. Moreover, all the edges in this path appear before e in the sorting. Thus, when Kruskal considers e, the nodes u and v are already connected (see the problem solved in class regarding a path consist with "cheap" edges). Thus, e is not added to the MST.

We conclude that this run of Kruskal produces an MST with edges only in E''.